

昨天 23:34

 **HW1(2).docx**
文档

给出连接ECE集群和AWS的具体操作步骤

 免费试用

 分享



根据你这份 **18-647 HW1 Part 2** 的要求，你需要在两类机器上运行同一份大整数乘法程序：

- 1. **ECE Community Compute Cluster**: `ece0XX.ece.local.cmu.edu`
- 2. **AWS EC2**: 选择一个 **1 vCPU** 的实例

作业明确要求两台机器都进行性能测试，并最终比较 ECE 和 AWS 的结果。  HW1

下面按你现在使用 **Windows + VS Code/Codespaces + Python** 的情况，给你一套可以直接照着做的流程。

一、连接 ECE 集群

Step 1：先确认你有 ECE cluster 权限

作业给出的机器格式是：

`ece0XX.ece.local.cmu.edu`

这里的 xx 不是固定的，要换成你被分配到的机器编号。作业本身只规定了 ECE number cluster machine，并没有在文件里给出你的具体机器号。 HW1

例如如果你被分配：

```
ece012.ece.local.cmu.edu
```



那么机器就是：

```
ece012.ece.local.cmu.edu
```



二、Windows 上 SSH 连接 ECE

你的 Windows 一般可以直接用 PowerShell。

打开：

```
PowerShell
```



输入：

```
</> PowerShell
```



```
ssh your_andrew_id@ece012.ece.local.cmu.edu
```

例如：

```
</> PowerShell
```



```
ssh wang6409@ece012.ece.local.cmu.edu
```

第一次连接可能出现：

```
The authenticity of host ... can't be established.  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```



输入：

```
yes
```



然后输入你的 CMU 密码。

注意：输入密码的时候，PowerShell **不会显示 *** 或任何字符，这是正常的。

成功后，你会看到类似：

```
[wang6409@ece012 ~]$
```



这就说明你已经进入 ECE 机器。

三、如果 ECE 只能通过 CMU VPN 访问

如果你在校外，最常见的问题是：

```
ssh: Could not resolve hostname ...
```



或者：

```
Connection timed out
```



因为：

```
ece0XX.ece.local.cmu.edu
```



属于 CMU 内部网络环境。

所以你需要先连接 **CMU VPN**，然后再：

```
</> PowerShell
```

```
ssh your_andrew_id@ece0XX.ece.local.cmu.edu
```



如果你已经在 CMU 网络环境下，可能不需要 VPN。

四、连接成功后，先检查机器

进入 ECE 后，先运行：

```
</> Bash
```

```
hostname
```



应该看到：

```
ece012
```



或者类似机器名。

再运行：

```
</> Bash
```

```
whoami
```



确认是你的账号。

然后：

```
</> Bash
```

```
python3 --version
```



以及：

```
</> Bash
```

```
nproc
```



查看 CPU 数量。

五、把你的 HW1 代码放到 ECE

假设你的本地项目结构：

```
HW1/  
├─ q1.py  
├─ q2.py  
└─ ...
```



最方便的方法之一是从 **本地 PowerShell** 使用 `scp` 。

例如：

</> PowerShell

```
scp .\q2.py your_andrew_id@ece012.ece.local.cmu.edu:~/q2.py
```



如果你的项目是整个文件夹：

</> PowerShell

```
scp -r .\HW1 your_andrew_id@ece012.ece.local.cmu.edu:~/
```



上传后登录：

</> PowerShell

```
ssh your_andrew_id@ece012.ece.local.cmu.edu
```



然后：

</> Bash

```
ls
```



应该看到：

```
HW1
```



进入：

```
</> Bash
```



```
cd HW1
```

六、如果你的代码是 GitHub repo

你现在本身就在使用 GitHub，那么其实更方便。

进入 ECE：

```
</> Bash
```



```
ssh your_andrew_id@ece012.ece.local.cmu.edu
```

然后：

```
</> Bash
```



```
git clone https://github.com/你的用户名/你的repo.git
```

例如：

```
</> Bash
```



```
git clone https://github.com/Wang6409/你的repo.git
```

然后：

```
</> Bash
```



```
cd 你的repo
```

检查：

```
</> Bash
```

```
git status
```

这样以后你本地修改代码：

```
Windows
```

```
↓ git push
```

```
GitHub
```

```
↓ git pull
```

```
ECE
```

会非常方便。

七、在 ECE 上运行你的 Python 程序

如果你的代码叫：

```
q2.py
```

直接：

```
</> Bash
```

```
python3 q2.py
```

如果需要参数，例如：

```
</> Bash
```

```
python3 q2.py 1024
```



就这样运行。

建议先测试：

```
</> Bash
```

```
python3 q2.py
```



确保程序能够正常运行。

八、记录 ECE 的实验结果

你的作业要求：

Start with strings of length 1, and double the length each time.

也就是：

```
1  
2  
4  
8  
16  
32  
64
```



```
128
256
512
1024
2048
4096
...
```

一直运行到:

- process 被 kill
- 或者某次运行超过 10 分钟

📄 HW1

所以建议程序把结果写到:

```
ece_results.txt
```



例如:

```
machine: ece012
algorithm: quadratic

n=1      time=...
n=2      time=...
n=4      time=...
...
```



然后你最终需要:

```
runs/
└─ ece_quadratic.txt
```



```
├─ ece_fft.txt
├─ aws_quadratic.txt
└─ aws_fft.txt
```

这也符合老师要求提交程序运行的 log 文件。  HW1

九、连接 AWS EC2

接下来创建 AWS EC2。

进入 [AWS EC2 官方页面](#) 。

登录 AWS Console 后：

```
EC2
↓
Instances
↓
Launch instance
```



十、选择 1 vCPU 实例

作业明确要求：

an AWS instance of your choice with one vCPU

 HW1

所以你选择实例时重点检查：

vCPU = 1



例如某些代际的：

t3.micro



是：

1 vCPU
2 GiB RAM



但**不要机械地只看实例名称**，因为 AWS 实例规格会变化；创建时在 EC2 控制台确认实际显示为 **1 vCPU**。

对于这份作业，内存也很重要，因为老师特别提醒：

memory may become an issue as the integers get larger

HW1

十一、创建 SSH Key Pair

在创建 EC2 时：

Key pair
→ Create new key pair



例如：

```
hw1-key
```



选择：

```
RSA
```



下载：

```
hw1-key.pem
```



这个 .pem 文件一定保存好。

例如：

```
C:\Users\你的用户名\.ssh\hw1-key.pem
```



十二、配置 Security Group

至少需要：

```
SSH  
TCP  
Port 22
```



如果只是自己 SSH 登录，不需要开放 HTTP 等端口。

对于 Source：

My IP

通常比：

Anywhere

更合适。

十三、启动 EC2

启动后进入：

EC2
→ Instances
→ 你的 instance

找到：

Public IPv4 address

例如：

3.XXX.XXX.XXX

假设是：

3.123.45.67

十四、Windows PowerShell 设置 PEM 权限

Windows 有时会因为 .pem 文件权限太宽而拒绝 SSH。

PowerShell 可以运行：

</> PowerShell

```
icacls "$env:USERPROFILE\.ssh\hw1-key.pem" /inheritance:r
```



然后：

</> PowerShell

```
icacls "$env:USERPROFILE\.ssh\hw1-key.pem" /grant:r "$($env:USERNAME):R"
```



十五、SSH 登录 AWS

Ubuntu EC2 通常使用：

</> PowerShell

```
ssh -i "$env:USERPROFILE\.ssh\hw1-key.pem" ubuntu@3.123.45.67
```



如果你选择的是 Amazon Linux，则用户名通常是：

ec2-user



对应：

```
</> PowerShell
```



```
ssh -i "$env:USERPROFILE\.ssh\hw1-key.pem" ec2-user@3.123.45.67
```

具体用户名取决于你选择的 AMI。

成功后类似：

```
ubuntu@ip-172-31-xx-xx:~$
```



这就说明已经进入 AWS。

十六、AWS 上检查 Python

运行：

```
</> Bash
```



```
python3 --version
```

然后：

```
</> Bash
```



```
which python3
```

例如：

```
/usr/bin/python3
```



如果需要安装依赖：

</> Bash

```
sudo apt update
```



然后根据你的程序需要安装。

如果你使用的是 NumPy FFT，例如：

</> Python

```
import numpy as np
```



▶ 运行

那么需要：

</> Bash

```
sudo apt install python3-numpy
```



或者使用 virtual environment。

十七、把代码放到 AWS

和 ECE 一样，有两种推荐方法。

方法 A: Git

AWS:

</> Bash



```
git clone https://github.com/你的用户名/你的repo.git
```

然后:

</> Bash



```
cd your-repo
```

方法 B: scp

Windows:

</> PowerShell



```
scp -i "$env:USERPROFILE\.ssh\hw1-key.pem" .\q2.py ubuntu@3.123.45.67:~/
```

然后 AWS:

</> Bash



```
ls
```

就可以看到:

q2.py



十八、在 AWS 上运行

例如：

```
</> Bash  
  
python3 q2.py
```



然后得到：

```
n=1  
n=2  
n=4  
n=8  
...
```



同样记录：

```
aws_results.txt
```



十九、建议你最终采用的目录结构

你的 HW1 最后可以整理成：

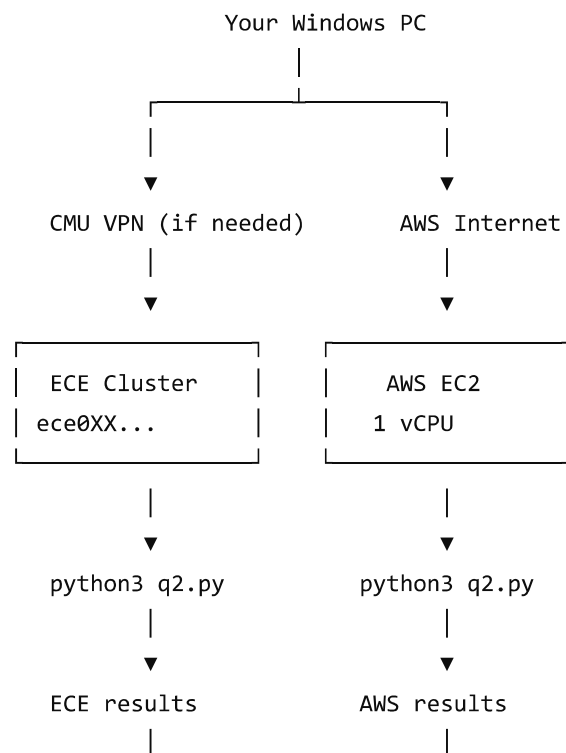
```
Lastname-hw1/  
|  
├─ src/  
|   └─ q2.py  
|  
├─ runs/  
|   ├── ece_quadratic.txt  
|   └─ ece_fft.txt
```

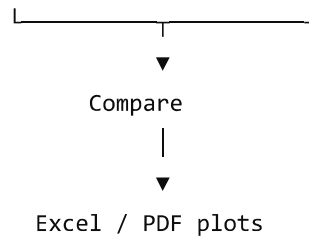


```
|   ├── aws_quadratic.txt
|   └── aws_fft.txt
|
|   ├── plots/
|   │   └── hw1_results.xlsx
|   └── ai/
|       └── chat_transcript.txt
```

作业要求的提交结构本身就是 `src/`、`runs/`、`plots/` 和 `ai/`，并最终打包成 `.zip`。📄 HW1

二十、整个流程可以记成这张图





最推荐你现在实际操作的顺序

先不要急着创建 AWS。

你可以按下面顺序做：

① 先连接 ECE

</> PowerShell

```
ssh your_andrew_id@ece0XX.ece.local.cmu.edu
```



② 确认成功

</> Bash

```
hostname
whoami
python3 --version
```



③ 上传你的 q2.py

</> PowerShell

```
scp .\q2.py your_andrew_id@ece0XX.ece.local.cmu.edu:~/
```



④ ECE 测试

```
</> Bash  
  
python3 q2.py
```



⑤ 再创建 AWS 1-vCPU EC2

然后:

```
</> PowerShell  
  
ssh -i "hw1-key.pem" ubuntu@<AWS-IP>
```



⑥ 上传同一个 q2.py

⑦ AWS 测试

```
</> Bash  
  
python3 q2.py
```



这样可以保证**同一份代码在 ECE 和 AWS 上运行**，之后才比较两台机器的性能。作业要求的最终图也是每个 problem size 用两个柱子比较 **ECE cluster vs AWS**。  HW1

如果你现在告诉我你的 **ECE 机器号** (例如 ece012) , 我可以直接给你一套从 **Windows PowerShell** → **SSH** → **上传** q2.py → **运行** → **保存 log** 的逐条命令。

来源











































































































































































































































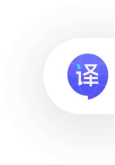












停止 实例

停止实例允许您降低成本、修改设置和对问题进行故障排除。

实例 ID ↗	停止保护	结果 ↗
 i-0ee89b64a516e2843	已禁用	 可以停止

▶ 已关联的资源

在实例停止期间，您仍将继续为这些资源支付费用

 您将为已关联的资源支付费用

您停止实例后，我们不再向您收取使用费或数据传输费。但是，您仍需要为关联的弹性 IP 地址和 EBS 卷支付费用。

跳过操作系统关闭

此选项将跳过正常的操作系统关闭过程。仅在必须立即停止实例时使用，例如在紧急情况或失效转移期间。

☐ 跳过操作系统关闭

取消

停止



今天 13:05

Run in terminal? ✕

```
Remove-Item -LiteralPath  
src\part2_python\__pycache__,src\part3_cuda\__pycache__  
-Recurse -Force -ErrorAction SilentlyContinue; git  
status --short
```

Allow ▾ Skip











































































































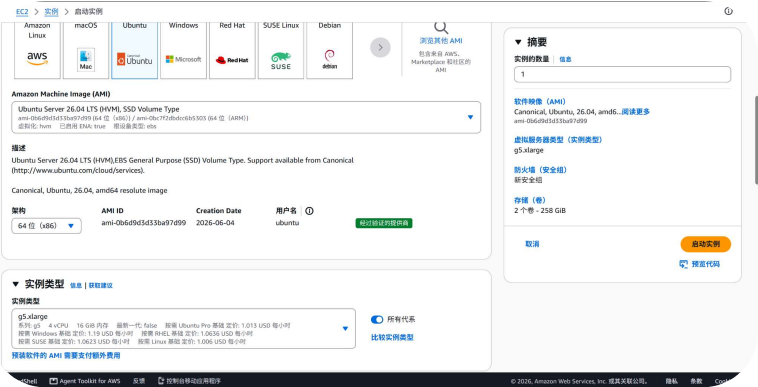












译

















































































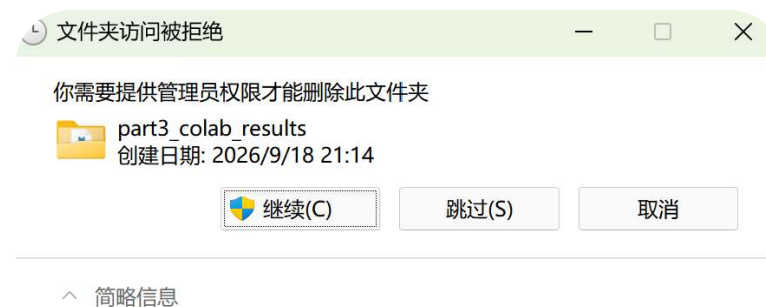












怎么办

这是 Windows 的**管理员权限提示**，不是 Colab 或 CUDA 的错误。你正在删除的文件夹 `part3_colab_results` 需要更高权限。

处理方法

方法一：直接点击“继续”

如果这是你自己刚刚生成、准备删除的文件夹：

1. 点击 **“继续(C)”**；
2. 如果弹出管理员确认窗口，选择**“是”**。

通常即可删除。

方法二：如果不想授权，点击“取消”

如果你不确定文件夹来源，先点击：

取消



不要点击“跳过”，因为“跳过”可能导致部分文件没有删除。

方法三：使用管理员 PowerShell 删除

如果点击“继续”仍然失败：

1. 关闭可能打开该文件夹的窗口、VS Code、Excel 等程序；
2. 在 Windows 开始菜单搜索：

PowerShell



3. 右键选择：

以管理员身份运行



4. 根据文件夹实际位置执行，例如：

</> PowerShell



```
Remove-Item -LiteralPath "C:\你的路径\part3_colab_results" -Recurse -Force
```

注意：把路径替换成文件夹的真实路径。

方法四：先取得文件夹所有权

如果仍提示拒绝访问，可以在管理员 PowerShell 中执行：

```
</> PowerShell  
  
takeown /F "C:\你的路径\part3_colab_results" /R /D Y
```



然后：

```
</> PowerShell  
  
icacls "C:\你的路径\part3_colab_results" /grant "$env:USERNAME:(OI)(CI)F" /T
```



再尝试删除：

```
</> PowerShell  
  
Remove-Item -LiteralPath "C:\你的路径\part3_colab_results" -Recurse -Force
```



我的建议

如果这个文件夹就是你刚刚从 Colab 下载的结果，而且你确认不再需要它，**直接点击“继续(C)”即可。**

如果你只是想重新下载一份结果，不必急着删除；也可以先将它重命名为：

```
part3_colab_results_old
```



然后保留备份。



做了更改，如何保存结果这不是 `nvidia-smi` 或 `cuFFT` 初始化失败；
`killed` 表示操作系统/Colab 运行时直接终止了进程，通常是内存压力。当前代码在未设置 `--max-length` 时会无限翻倍，直到下一次规模的 CPU/GPU 分配把 Colab 内存耗尽；这种 `SIGKILL` 无法被 C++ 的异常处理捕获，所以不会写出 `oom` 行。我会加入 GPU 内存预检查和安全停止，让它在 OOM 前记录状态并正常结束。

Edit , ,
Completed with input:
{
 "path": "e:\\CMU\\3rd_semester\\18647\\HWs\\647_hw1\\README.md"
}

展开 ∨

你这次修改的是本地仓库中的代码。要让 Colab 使用新代码并保存新的实验结果，需要分成两部分：

1. 保存并推送代码修改到 GitHub；
2. 在 Colab 拉取新代码、重新编译并运行实验，下载结果。

一、先在本地保存代码修改

在 Windows PowerShell 中进入项目目录：

</> PowerShell



```
cd E:\CMU\3rd_semester\18647\Hws\647_hw1
```

查看修改:

```
</> PowerShell
```

```
git status
```



确认修改内容:

```
</> PowerShell
```

```
git diff -- README.md src/part3_cuda/bigint_fft.cu src/part3_cuda/main.cu
```



如果修改正确, 提交并推送:

```
</> PowerShell
```

```
git add README.md src/part3_cuda/bigint_fft.cu src/part3_cuda/main.cu
git commit -m "Add memory checks and safe CUDA experiment termination"
git push origin main
```



如果你还修改了其他 Part 3 文件, 可以直接:

```
</> PowerShell
```

```
git add .
git commit -m "Improve CUDA OOM handling and Colab workflow"
git push origin main
```



确认 GitHub 已经更新后，再进入 Colab。

二、在 Colab 获取最新代码

如果当前 Colab 目录已经存在：

</> Python

```
%cd /content/18647_hw1  
!git pull origin main
```



▶ 运行

如果尚未克隆：

</> Python

```
!git clone https://github.com/Wang6409/18647_hw1.git /content/18647_hw1  
%cd /content/18647_hw1
```



▶ 运行

确认修改已经拉取：

</> Python

```
!grep -n "cudaMemGetInfo\\|MemAvailable\\|oom" src/part3_cuda/bigint_fft.cu
```



▶ 运行

如果能看到相关代码，说明新版本已经在 Colab 中。

三、重新编译，不能直接运行旧的可执行文件

这是非常重要的一步。

</> Python



▶ 运行

```
%cd /content/18647_hw1
!make -C src/part3_cuda clean
!make -C src/part3_cuda
```

检查编译结果：

</> Python



▶ 运行

```
!ls -lh src/part3_cuda/fft_multiply
```

如果看到 `fft_multiply`，说明新程序已经生成。

四、先运行 smoke test

不要直接开始十分钟实验，先验证新代码能够正常结束并生成 CSV。

</> Python



▶ 运行

```
!mkdir -p runs plots
```

</> Python



▶ 运行

```
!src/part3_cuda/fft_multiply \
  --max-length 8 \
  --output runs/part3_smoke.csv
```

查看结果：

</> Python



▶ 运行

```
!cat runs/part3_smoke.csv
```

检查是否有：

- success ;
- 正确的运行时间；
- 没有 CUDA 错误；
- 没有 Segmentation fault ；
- 没有直接出现 Killed 。

如果 smoke test 正常，再运行正式实验。

五、运行正式实验并保存日志

推荐不要只执行普通命令：

</> Python



▶ 运行

```
!bash src/part3_cuda/run_colab.sh
```

而是使用 tee ， 同时在屏幕显示并保存完整日志：

</> Python



▶ 运行

```
!bash src/part3_cuda/run_colab.sh 2>&1 | tee runs/part3_colab_manual.log
```

不过要注意：如果 `run_colab.sh` 内部已经将日志写入：

```
runs/part3_colab.log
```



那么最终可能同时存在两个日志文件：

```
runs/part3_colab.log  
runs/part3_colab_manual.log
```



这没有问题，但提交时可以保留一个完整、清晰的版本。

正式运行时不要设置 `--max-length`，让程序根据新的内存检查逻辑自动停止：

</> Python



▶ 运行

```
!bash src/part3_cuda/run_colab.sh
```

六、检查新代码是否正确记录停止原因

实验结束后查看原始 CSV：

</> Python



▶ 运行

```
!cat runs/gpu_results.csv
```

重点观察最后几行：

</> Python



▶ 运行

```
!tail -10 runs/gpu_results.csv
```

如果内存预检查生效，最后可能出现类似：

```
...,oom:insufficient GPU memory for next problem size
```



或者：

```
...,oom:insufficient host memory for next problem size
```



这表示程序主动检测到内存不足并停止，而不是被系统直接 Killed。

如果最后仍然只有：

```
Killed
```



说明：

- 可能运行的仍是旧可执行文件；
- 修改没有成功拉取；
- 内存预检查没有覆盖实际发生内存压力的位置；
- 或者 Colab 运行时在预检查之前就终止了进程。

此时先检查：

</> Python



▶ 运行

```
!grep -n "cudaMemGetInfo\\|MemAvailable\\|oom" src/part3_cuda/bigint_fft.cu
```

并确认重新执行过：

</> Python



▶ 运行

```
!make -C src/part3_cuda clean  
!make -C src/part3_cuda
```

七、保存实验环境信息

即使脚本已经生成环境文件，也建议单独保存一次：

</> Python



▶ 运行

```
!nvidia-smi > runs/part3_environment.txt  
!echo "----- CUDA compiler -----" >> runs/part3_environment.txt  
!nvcc --version >> runs/part3_environment.txt  
!echo "----- CPU -----" >> runs/part3_environment.txt  
!lscpu >> runs/part3_environment.txt
```

查看：

</> Python



▶ 运行

```
!cat runs/part3_environment.txt
```

八、生成表格和 PDF

如果 `run_colab.sh` 已经自动生成，可以直接检查：

</> Python



▶ 运行

```
!find runs plots -maxdepth 1 -type f -print
```

预期包括：

```
runs/gpu_results.csv  
runs/gpu_fft_table.csv  
runs/part3_colab.log  
runs/part3_environment.txt  
plots/gpu_fft_mops.pdf
```



如果没有生成表格或 PDF，先查看绘图脚本的实际参数：

</> Python



▶ 运行

```
!python src/part3_cuda/plot_results.py --help
```

然后按照帮助信息运行，不要盲目使用之前的参数。

九、把结果压缩保存到本地

确认结果存在：

</> Python



▶ 运行

```
!ls -lh runs
!ls -lh plots
```

然后压缩：

</> Python



▶ 运行

```
!zip -r /content/part3_colab_results.zip \
    src/part3_cuda \
    runs \
    plots \
    README.md
```

检查压缩包：

</> Python



▶ 运行

```
!ls -lh /content/part3_colab_results.zip
```

在 Colab 左侧文件栏中找到：

```
/content/part3_colab_results.zip
```



右键选择 **Download** 下载到 Windows。

十、如果想直接下载单个文件

也可以在 Colab 中使用：

</> Python



▶ 运行

```
from google.colab import files

files.download('/content/part3_colab_results.zip')
```

注意：下载后文件会保存到浏览器默认下载目录，通常是 Windows 的：

C:\Users\你的用户名\Downloads



如果浏览器询问是否允许多个下载或下载文件，选择允许。

十一、推荐最终保存的文件

至少保留以下内容：

```
part3_colab_results/
├── src/
│   └── part3_cuda/
│       ├── main.cu
│       ├── bigint_fft.cu
│       ├── bigint_fft.h
│       ├── utils.cu
│       ├── utils.h
│       ├── Makefile
│       ├── run_colab.sh
│       └── plot_results.py
└── runs/
```



```
|   |— part3_smoke.csv  
|   |— gpu_results.csv  
|   |— gpu_fft_table.csv  
|   |— part3_colab.log  
|   |— part3_environment.txt  
|   |— part3_colab_manual.log  
|— plots/  
|   |— gpu_fft_mops.pdf  
|— README.md
```

如果使用了 AI，还需要另外保存：

```
ai/  
└─ AI usage transcript or prompt record
```



最简操作流程

本地 PowerShell

```
</> PowerShell  
  
cd E:\CMU\3rd_semester\18647\Hws\647_hw1  
git add .  
git commit -m "Add memory checks and safe CUDA experiment termination"  
git push origin main
```



Colab

</> Python



▶ 运行

```
%cd /content/18647_hw1
!git pull origin main
!make -C src/part3_cuda clean
!make -C src/part3_cuda
```

</> Python



▶ 运行

```
!mkdir -p runs plots
!src/part3_cuda/fft_multiply \
    --max-length 8 \
    --output runs/part3_smoke.csv
!cat runs/part3_smoke.csv
```

确认 smoke test 正常后:

</> Python



▶ 运行

```
!bash src/part3_cuda/run_colab.sh 2>&1 | tee runs/part3_colab_manual.log
```

最后:

</> Python



▶ 运行

```
!zip -r /content/part3_colab_results.zip \
    src/part3_cuda runs plots README.md
```

然后下载:

</> Python

from google.colab import files

files.download('/content/part3_colab_results.zip')

运行

关键点：代码修改要先 git push , Colab 必须 git pull , 然后执行 make clean 和重新 make ; 否则 Colab 可能仍然运行旧的 fft_multiply 。





Atlassian

广告 ...

Give agents a plan. Not a prompt.
You bring the idea. Jira provides the context, specs, and guardrails agents...

今天 23:40

如何导出这个chatgpt ai使用记录

如果你要导出当前这个 ChatGPT 对话，作为 CMU 作业的 AI 使用记录，有两种方法。

方法一：导出整个 ChatGPT 数据（最完整）

适合需要保存完整对话、代码、附件和历史记录的情况。

- 1. 点击 ChatGPT 左下角的个人头像/账户菜单；
- 2. 进入 Settings (设置) ；
- 3. 选择 Data Controls (数据控制) ；

4. 找到 **Export Data** (导出数据) ;
5. 点击 **Export**;
6. 在确认窗口点击 **Confirm export**。

之后, OpenAI 会向你的账户邮箱或手机发送导出通知。下载链接可能需要等待, 官方说明最长可能需要 **7 天**, 收到邮件后链接通常在 **24 小时后过期**。下载结果是一个 ZIP 文件, 里面包含聊天记录及其他账户数据。

 OpenAI Help Center

下载后如何找到聊天记录

解压 ZIP 文件, 通常可以找到:

conversations.json



较大的导出文件也可能被分成多个 conversation JSON 文件。导出内容可能还包含一个可浏览的 HTML 聊天记录文件。  OpenAI Help Center +1

方法二: 只保存当前这个对话 (更适合交作业)

如果老师只需要你这次 Part 3 的 AI 使用记录, 不一定要导出整个账户。

方案 A: 打印成 PDF

在当前对话页面:

1. 按 **Ctrl + P** ;
2. 打印机选择:

Save as PDF



3. 点击 **Save**;

4. 文件名可以设为:

AI_Usage_Record_Part3_ChatGPT.pdf



这是最简单、最适合提交给老师的方式。

建议保存时包含:

- 你的提问;
- ChatGPT 的回答;
- 代码修改建议;
- Colab 运行步骤;
- OOM / Killed 错误分析;
- 结果保存和导出方法。

方案 B: 复制到 Word

1. 从当前对话开始处选择内容;
2. 复制;
3. 粘贴到 Word 或 Google Docs;
4. 添加标题:

AI Usage Record - CMU 18-647 Part 3



5. 保存为:

AI_Usage_Record_Part3.docx



也可以另存为 PDF。

推荐的提交文件结构

如果作业要求 ai/ 文件夹，可以整理成：

```
ai/  
├── AI_Usage_Record_Part3.pdf  
└── AI_Prompts_Part3.txt
```



其中：

AI_Usage_Record_Part3.pdf

保存完整对话，建议使用浏览器打印成 PDF。

AI_Prompts_Part3.txt

可以列出你实际使用过的主要问题，例如：

1. Summarize the required operations for Part 3 after modifying the CUDA code.
2. Explain how to run the CUDA program in Google Colab with a GPU.
3. Explain the cause of the "Killed" error during the CUDA experiment.
4. Suggest how to add GPU and CPU memory checks before allocation.



5. Explain how to save and export the experiment results.

注意：如果课程要求完整 AI transcript，最好保存完整对话，而不只是整理后的 prompt 列表。

我的建议

对于你这次作业，最实用的方式是：

- 1. 用 Ctrl + P → Save as PDF 保存当前对话；
- 2. 文件命名为：

AI_Usage_Record_Part3_ChatGPT.pdf



3. 放入：

ai/



4. 如果老师还要求 prompts，再额外创建：

AI_Prompts_Part3.txt



这样比导出整个 ChatGPT 账户数据更方便，也更容易让老师查看。



ChatGPT 也可能会犯错。请核查重要信息。

